

Nexus Project Report

Objective

The objective of this project was to build the same login application using a common Next.js frontend with two different Python backends: Flask and FastAPI. Both applications used the same SQLite database and implemented identical login functionality.

Project Structure

Nexus/

■■■ nextjs-flask/ (Next.js + Flask + SQLite)

■■■ nextjs-fastapi/ (Next.js + FastAPI + SQLite)

Comparison

Flask

- Easier to set up and understand.
- Simple syntax and beginner-friendly.
- Suitable for small web applications.

FastAPI

- Slightly more setup (requires Uvicorn and Pydantic).
- Automatic request validation.
- Automatic Swagger API documentation (/docs).
- Better performance due to ASGI and async support.

Performance

For this login application, both backends performed well. However, FastAPI generally offers better performance than Flask because it is built on ASGI and supports asynchronous request handling. The difference is minimal for small projects but becomes more significant as applications scale.

Advantages

Flask

- Easy to learn
- Less configuration
- Large community support

FastAPI

- Faster execution
- Built-in API documentation
- Automatic data validation
- Better suited for scalable APIs

Conclusion

Flask was easier to set up and is recommended for beginners and small projects. FastAPI required a little more configuration but provided better performance, automatic API documentation, and built-in validation. For modern, scalable API development, FastAPI is the stronger choice.